Resume Q&A Project Documentation

Oct 10, 2026 · @RAGHU

Overview

This project answers questions about a resume by searching a small set of Markdown notes for the closest match. You ask a question in plain English, and it returns the note section that best fits it, with a similarity score.

It is built in a Python notebook (Google Colab or Jupyter) with these tools:

- **sentence-transformers**: turns text into vectors (embeddings) using the all-MiniLM-L6-v2 model
- **numpy**: compares vectors with a dot product
- **os, re, glob**: read, check, and write the note files

The notes are stored under a `knowledge/` folder, so you can add new facts by adding or editing files, without changing the code.

Knowledge base

Each note is a Markdown file with a short header and one or more `##` sections. The project creates two sample notes and an index:

```
knowledge/
├── index.md              (generated list of notes)
└── resume/
    ├── education.md      (type: Resume)
    └── projects.md       (type: Project)
```

Every note must follow this format, or the validation step rejects it:

```
---
type: Resume
title: Education
---
# Education
## Degree
I am pursuing B.Tech in Computer Science and Artificial Intelligence (CAI).
```

Requirement	Rule checked	Why it matters
Type	A `type:` line in the header	Labels what kind of note it is
Title	One `#` heading	Prefixed to every section so search has context
Section	At least one `##` heading	Each section becomes one searchable piece

Sample notes included

Note	Sections	Content
education.md	Name, Degree, College, Diploma	Name, B.Tech in Computer Science and AI, college, and diploma score of 83.75%
projects.md	Purpose, Technologies, Java Role, Python Role, Data Files	Insurance Fraud Claim Scout: flags suspicious claims using Java rules and a Python risk score

The files `index.md` and `log.md` are skipped when notes are read, because they are not content notes.

Run order

Run the cells in this order. The notes cell must run before the validation cell, even though it appears last in the pasted code.

1. **Setup**: install sentence-transformers and import the libraries.
2. **Create notes**: run the `NOTES` cell to write `education.md` and `projects.md` into `knowledge/resume/`.
3. **Validate and index**: check every note, then write `knowledge/index.md`. Fix any note marked with a red cross.
4. **Build search**: split notes into sections, embed them, and define `find()`.
5. **Ask questions**: call `find("your question")` and read the result.

To add knowledge later, add a new `.md` file under `knowledge/`, then re-run steps 3 and 4.

Code walkthrough

1. Setup

Installs `sentence-transformers` quietly and imports `os`, `re`, `glob`, and `numpy`. It prints a confirmation when ready.

2. Create notes (the `NOTES` dictionary)

Maps each file path to its Markdown text. A loop creates the folder with `os.makedirs`, writes each file in UTF-8, and prints `Saved:` for each one.

3. Validate notes and build the index

- `glob.glob("knowledge/**/*.md", recursive=True)` finds every note, skipping `index.md` and `log.md`.
- Three regular expressions check for a `type:` line, a `#` title, and a `##` section.
- Each note prints a green check, or a red cross listing what is missing.
- A new `knowledge/index.md` is written with a link line for every valid file.
- The final message reports how many notes were checked and how many need fixes.

4. Build the search engine

`load_pieces()` reads each note, takes its title, and splits the text at every `##` heading. Each section becomes one piece, stored as `"<title>: <section text>"`. Prefixing the title keeps meaning when a section is short, such as Diploma under Education.

The model `all-MiniLM-L6-v2` encodes all pieces into `VECTORS`. Using `normalize_embeddings=True` makes each vector length 1, so a dot product equals cosine similarity.

`find(question)` encodes the question, scores it against every piece (`VECTORS @ question_vector`), and returns the highest score with its piece.

Function or variable	Purpose
`files`	List of valid note paths, created in the validation cell
`PIECES`	List of dictionaries with `file` and `text` for every section
`VECTORS`	Matrix of normalized embeddings, one row per piece

Function or variable	Purpose
`find(question)`	Returns (score, piece) for the best match

Example

The notebook tests the search with a question about the diploma score:

```
score, piece = find("What is my diploma percentage?")

print("Found in:", piece["file"])
print("Matching score:", round(score, 2))
print("Information:", piece["text"])
```

The best match should be the Diploma section of the education note, which reads: "Education: Diploma I completed my Diploma with 83.75%." The exact score depends on the model, so run it to see your own value.

Reading the score

Because vectors are normalized, the score is cosine similarity, between -1 and 1. Higher means a closer match. A score near 1 means a strong match; a low score means the notes probably do not contain the answer.

More questions to try

- "Which college do I study at?"
- "What technologies does the fraud project use?"
- "What does Python do in the project?"
- "Which data files are used?"

Limitations and next steps

Limitation	Suggested fix
`find()` always returns a best match, even when no note answers the question	Add a minimum score, such as 0.3, and reply "not found" below it
Only the top result is returned	Return the top 3 using `np.argsort` for more context

Limitation	Suggested fix
Notes are embedded again every time the notebook starts	Save `VECTORS` with `np.save` and reload it
Cells depend on each other (`files` is created in the validation cell)	Wrap the steps in functions, or restart and run all cells in order
The note files are rewritten on every run	Skip writing a note if the file already exists
The answer is the raw note text	Add a short step that turns the section into a full sentence

Privacy: the notes contain personal details such as your name, college, and scores. Keep the notebook and `knowledge/` folder private if you share the project.

Full source code

The complete notebook, in the order the cells should run.

Cell 1: Setup

```
%pip install -q sentence-transformers

import os
import re
import glob
import numpy as np

print("✅ Resume Q&A project setup ready!")
```

Cell 2: Create the notes

```
NOTES = {
    "knowledge/resume/education.md": """---
type: Resume
title: Education
---
# Education
## Name
My name is Akula Hema Priya.
## Degree
I am pursuing B.Tech in Computer Science and Artificial Intelligence (CAI).
```

```
## College
I study at Kakinada Institute of Engineering and Technologies (KIET).
## Diploma
I completed my Diploma with 83.75%.
""",

    "knowledge/resume/projects.md": """---
type: Project
title: Insurance Fraud Claim Scout
---
# Insurance Fraud Claim Scout
## Purpose
The project flags potentially suspicious insurance claims for review.
## Technologies
The project uses Java and Python.
## Java Role
Java processes claims and applies rules based on previous claims,
claim amount, and policy average.
## Python Role
Python calculates a ratio-based fraud risk score.
## Data Files
The project uses claims.csv and fraud_scores.csv.
"""
}

for path, text in NOTES.items():
    os.makedirs(os.path.dirname(path), exist_ok=True)
    with open(path, "w", encoding="utf-8") as file:
        file.write(text)
    print("Saved:", path)

print("\n✅ Resume notes ready!")
```

Cell 3: Validate the notes and write the index

```
files = [
    path
    for path in sorted(glob.glob("knowledge/**/*.md", recursive=True))
    if os.path.basename(path) not in ("index.md", "log.md")
]

problems = 0
```

```
for path in files:
    with open(path, "r", encoding="utf-8") as file:
        text = file.read()

    missing = []

    if not re.search(r"^type:\s*\S+", text, re.M):
        missing.append("type")
    if not re.search(r"^# .+", text, re.M):
        missing.append("title")
    if not re.search(r"^## .+", text, re.M):
        missing.append("section")

    if missing:
        problems += 1
        print("❌", path, "Missing:", ", ".join(missing))
    else:
        print("✅ OK:", path)

index_text = """---
type: Index
title: Resume Knowledge Index
---
# Resume Knowledge Index
"""

for path in files:
    index_text += f"- {path.removeprefix('knowledge/')}\n"

with open("knowledge/index.md", "w", encoding="utf-8") as file:
    file.write(index_text)

print(f"\nChecked {len(files)} notes.")
print("✅ All notes are good!" if problems == 0
      else f"❌ Fix {problems} note(s) before continuing.")
```

Cell 4: Build the search and test it

```
from sentence_transformers import SentenceTransformer

def load_pieces():
    pieces = []
```

```
    for path in files:
        with open(path, "r", encoding="utf-8") as file:
            text = file.read()

        title = re.search(r"^# (.+)$", text, re.M).group(1)

        sections = re.split(r"^## ", text, flags=re.M)[1:]

        for section in sections:
            pieces.append({
                "file": path,
                "text": f"{title}: {section.strip()}"
            })

    return pieces

PIECES = load_pieces()

if not PIECES:
    raise ValueError("No resume sections found. Check Cells 2 and 3.")

model = SentenceTransformer("all-MiniLM-L6-v2")

VECTORS = model.encode(
    [piece["text"] for piece in PIECES],
    normalize_embeddings=True
)

def find(question):
    question_vector = model.encode(
        [question],
        normalize_embeddings=True
    )[0]

    scores = VECTORS @ question_vector
    best_index = int(np.argmax(scores))

    return float(scores[best_index]), PIECES[best_index]

print("✅ Search ready!")
print("Total sections:", len(PIECES))

score, piece = find("What is my diploma percentage?")
```

```
print("\nFound in:", piece["file"])
print("Matching score:", round(score, 2))
print("Information:", piece["text"])
```